CRYPTO HAWKING · ETHDENVER 2025 WINNER

Smart Contract Security Audit

FermiSwapper

0xb1076fE3AB5e28005C7c323Bac5AC06a680d452e

Ethereum

Compiler: v0.8.28+commit.7893614a

Date: October 11, 2026

OVERALL RISK RATING AI-assisted audit. Not a substitute for manual review.

High

Executive Summary

Independent security review of publicly verified source, not commissioned by the project. FermiSwapper is a compact inventory based swap router: quotes and accounting are delegated to an external FermiEngine, token inventory lives in a separate traderVault, and users route swaps through this contract after granting it an ERC-20 allowance. The swap logic itself is tight, reentrancy guards are present on both entry points, and the callback variant verifies actual received balances rather than trusting the caller. The risk sits almost entirely in the privileged surface inherited from BaseManager. A single immutable manager key can execute arbitrary calls from this contract, which converts every user allowance granted to the router into funds that key can move at will. The manager address cannot be rotated, so a leaked key cannot be revoked by the protocol, only by every user individually revoking approvals. Nothing here is broken in normal operation; the authority is simply unbounded.

Findings (10)

F-001

High

Arbitrary call capability turns every user allowance into manager controlled funds

contracts/BaseManager.sol:69–70

DESCRIPTION

BaseManager.executeGeneral performs target.call(callData) with both arguments fully controlled by the manager, with no allowlist of targets and no restriction on the selector. FermiSwapper pulls input tokens from the user with safeTransferFrom(msg.sender, traderVault, amountIn), so any user who swaps must first approve this contract, and wallets commonly request an unlimited approval. The manager can therefore call executeGeneral(token, abi.encodeWithSelector(IERC20.transferFrom.selector, victim, attacker, allowance)) and move any approving user's balance up to their remaining allowance, without touching the swap path at all.

IMPACT

A compromised or malicious manager key can drain every outstanding allowance held against this router, across every token any user has ever approved. The loss is bounded only by the sum of live approvals, which for unlimited approvals is each user's full token balance. This requires privileged access, so it is a key compromise and trust risk rather than an openly exploitable bug, but the blast radius is the entire user base rather than the contract's own holdings.

RECOMMENDATION

Remove executeGeneral and executeGeneralMulti if they are only used for occasional recovery, and replace them with narrow, purpose built functions (for example a sweep that can only move the contract's own balances to the vault). If a general escape hatch must remain, restrict it with an allowlist of target addresses and an explicit denylist of the transferFrom and approve selectors, put it behind a timelock, and emit an event for every call. Users should also be encouraged to approve exact amounts rather than unlimited.

```
function executeGeneral(address target, bytes calldata callData)
public
onlyManager
returns (bool, bytes memory)
{
return target.call(callData);
}
```

F-002

Medium

Manager key is immutable, so a compromise cannot be revoked on-chain

contracts/BaseManager.sol:11–14

DESCRIPTION

BaseManager stores manager as an immutable address set at construction. There is no transferManager, no two step handover and no pause. The vault address can be changed, the engine can be changed, but the account that controls all of those cannot. Combined with F-001, this means the single most powerful key in the system is also the only one that can never be rotated.

IMPACT

If the manager key is lost, the protocol loses the ability to move inventory, retire the router or respond to an incident. If the key is stolen, there is no on-chain action the team can take to stop the attacker. The only remaining defence is asking every user to revoke their approvals individually, which in practice reaches a small fraction of holders.

RECOMMENDATION

Make the manager role transferable through a two step process (propose, then accept) so a key can be rotated proactively and so a mistyped address cannot brick the system. Move the role to a multisig rather than an externally owned account, and add a pause that halts both swap entry points so an incident can be contained while users revoke.

```
address payable private immutable manager;

modifier onlyManager() {
require(msg.sender == manager, "NM"); // Not Manager
_;
}
```

F-003

Medium

Pricing engine can be replaced with an arbitrary contract

contracts/FermiSwapper.sol:33–34

DESCRIPTION

setFermi accepts any address whose traderVault() happens to return the configured vault, and that address then becomes the source of both quote() and swap(). The check confirms a matching vault but proves nothing about the pricing logic, so a replacement engine can return any amountIn and amountOut it likes. The per swap amountCheck parameter does protect a user who sets it correctly, since the router reverts when the result is worse than their limit, but any integrator that passes a loose or zero amountCheck is fully exposed to whatever the engine returns.

IMPACT

A manager who swaps in a hostile engine can extract value from every caller that does not enforce its own slippage bound, including aggregators and scripts that pass amountCheck as zero. Because the swap path still looks and behaves normally, the manipulation is difficult to notice from the outside.

RECOMMENDATION

Put engine changes behind a timelock so integrators can observe a pending change before it takes effect, and emit an event carrying both the previous and the new address. Consider sanity bounds enforced in the

router itself, for example a maximum deviation between the engine quote and a reference oracle, so a single component cannot define price unilaterally.

```
function setFermi(address newFermi) external onlyManager {
require(newFermi != address(0), "ZA");
require(IFermiEngine(newFermi).traderVault() == traderVault, "TVM");
fermi = IFermiEngine(newFermi);
}
```

F-004

Medium

Output transfer assumes exact delivery, so fee on transfer tokens short the recipient

contracts/FermiSwapper.sol:70–71

DESCRIPTION

In both swap paths the output leg is a plain safeTransferFrom(traderVault, recipient, amountOut) and the result is never measured. If tokenOut charges a transfer fee or rebases, the recipient receives less than amountOut while the accounting, the emitted event and the caller's amountCheck all record the full amount. The input leg of the callback variant is handled correctly, since it compares real balances before and after, which makes the asymmetry between the two legs the more likely oversight.

IMPACT

Users and integrators of fee bearing pairs receive less than the contract reports, and off-chain systems that trust the FermiSwap event will drift from on-chain reality. The slippage check gives no protection here because it validates the quoted figure, not the delivered one.

RECOMMENDATION

Measure the recipient's balance before and after the output transfer and either revert when the delivered amount is below the checked amount, or report the delivered amount in the event and return value. Alternatively, document the pair allowlist as excluding fee on transfer and rebasing tokens, and enforce that when pairs are registered.

```
(amountIn, amountOut) = fermi.swap(tokenIn, tokenOut, amountSpecified, msg.sender);
require(amountSpecified > 0 ? amountOut >= amountCheck : amountIn <= amountCheck,
"ACF");

IERC20(tokenOut).safeTransferFrom(traderVault, recipient, amountOut);
```

F-005

Low

Callback window leaves the system in a visibly inconsistent state

contracts/FermiSwapper.sol:93–94

DESCRIPTION

fermiSwapWithCallback delivers tokenOut to the recipient, then hands control to the caller through fermiSwapCallback, and only afterwards collects tokenIn. The nonReentrant guard prevents re-entry into this router, but it says nothing about the engine or the vault, which remain callable by the untrusted callback while the accounting has already been updated and the input has not yet arrived. Whether this is exploitable depends on FermiEngine, which is outside the scope of this review.

IMPACT

Any logic in the engine or vault that reads balances, inventory or quotes during that window observes a state where output has left but input has not been received. If quoting depends on vault balances, a second swap performed inside the callback could be priced against a temporarily depleted inventory.

RECOMMENDATION

Confirm that FermiEngine and the trader vault carry their own reentrancy protection, or add a shared lock so the whole flow is atomic from the engine's point of view. Documenting the invariant the engine relies on would also make future integrations safer.

```
IERC20(tokenOut).safeTransferFrom(traderVault, recipient, amountOut);
IFermiSwapCallback(msg.sender).fermiSwapCallback(int256(amountIn),
-int256(amountOut), callbackData);
require(IERC20(tokenIn).balanceOf(address(this)) >= balanceBefore + amountIn, "WTA");
```

F-006

Low

Overpaid input is kept by the contract rather than refunded

contracts/FermiSwapper.sol:95–96

DESCRIPTION

The callback path requires the contract's tokenIn balance to grow by at least amountIn, then forwards exactly amountIn to the trader vault. Anything sent above that stays in the router, where only the manager can move it, and it goes to the vault rather than back to the payer.

IMPACT

Integrators that round up, or that send the full balance during the callback, silently lose the excess. The amounts are usually small, but the path is one way and there is no user facing recovery.

RECOMMENDATION

Return any surplus above amountIn to msg.sender at the end of the callback path, or revert when the received amount exceeds amountIn so the caller corrects the integration rather than losing funds quietly.

```
require(IERC20(tokenIn).balanceOf(address(this)) >= balanceBefore + amountIn, "WTA");
IERC20(tokenIn).safeTransfer(traderVault, amountIn);
```

F-007

Low

Privileged configuration changes emit no events

contracts/FermiSwapper.sol:39–40

DESCRIPTION

setFermi, setTraderVault and setVaultAddress all change security critical routing with no event. The only on-chain record is the transaction itself, which off-chain monitoring has to discover by decoding calldata rather than by subscribing to logs.

IMPACT

Integrators, dashboards and incident monitoring cannot react to a pricing engine swap or a vault change in real time. In an incident, reconstructing when a change happened requires a full transaction scan of the contract.

RECOMMENDATION

Emit a dedicated event on each setter carrying the previous and the new value, following the pattern already used for FermiSwap. This costs very little gas and is the difference between a change that is noticed in seconds and one noticed after the fact.

```
function setTraderVault(address payable newTraderVault) external onlyManager {
require(newTraderVault != address(0), "ZA");
require(fermi.traderVault() == newTraderVault, "TVM");
traderVault = newTraderVault;
// no event emitted
}
```

F-008

Low

executeGeneral ignores call failure and calls to addresses with no code

contracts/BaseManager.sol:69–74

DESCRIPTION

The single call variant returns the success flag to the caller instead of reverting on it, while the multi call variant does revert. A low level call to an address with no deployed code also returns success true, so a mistyped or not yet deployed target reports success and does nothing. Only the multi path performs any check.

IMPACT

Operational rather than financial: a failed recovery or configuration call can appear to succeed, and the discrepancy only becomes visible later when the expected state change is missing.

RECOMMENDATION

Revert on failure in the single call path as well, and require target.code.length > 0 before calling. Keeping both paths consistent removes a class of silent operational error.

```
function executeGeneral(address target, bytes calldata callData)
public onlyManager returns (bool, bytes memory)
{
return target.call(callData);
}
```

F-009

Informational

Floating pragma with no upper bound

contracts/FermiSwapper.sol:2–undefined

DESCRIPTION

Both files declare pragma solidity >=0.8.25 with no upper bound, while the deployed bytecode was produced by 0.8.28. A lower bound alone allows the same source to compile under future compiler versions whose behaviour has not been tested here.

IMPACT

No effect on the deployed contract. It matters for reproducibility: anyone verifying or redeploying from this source may silently use a different compiler than the one reviewed.

RECOMMENDATION

Pin the exact version used for production builds, for example pragma solidity 0.8.28, so source, bytecode and review always refer to the same compiler.

```
pragma solidity >=0.8.25;
```

F-010

Informational

Open receive and fallback accept ETH that only the manager can move

contracts/FermiSwapper.sol:101–102

DESCRIPTION

Both receive and fallback are payable and empty, so the contract silently accepts ETH from anyone, including through a fallback call with arbitrary calldata. The only exit is transferEth or withdrawAllEth, both manager only, and both send to the vault rather than back to the sender.

IMPACT

ETH sent by mistake is not recoverable by the sender. The amounts are usually small, but there is no user facing path to recover them and no event recording the receipt.

RECOMMENDATION

If the router does not need to hold ETH, remove both functions so mistaken transfers revert. If they are needed for an integration, emit an event on receipt so the origin of any balance can be traced.

```
receive() external payable {}
fallback() external payable {}
```

Gas Optimizations

Cache the traderVault read in the swap paths

contracts/FermiSwapper.sol:73–93

Both swap functions read the traderVault storage slot once and the fermi slot once per call. Loading each into a local variable at the start of the function saves a warm SLOAD on the second use in the callback path and makes the data flow easier to follow.

Use unchecked increment in the withdrawal loop

contracts/BaseManager.sol:51–52

withdrawAllTokens iterates with ++i over a calldata array whose length cannot overflow. Wrapping the increment in an unchecked block removes a redundant overflow check per iteration, which is noticeable when sweeping a long token list.

Use custom errors instead of short require strings

contracts/FermiSwapper.sol:26–27

The codebase uses abbreviated revert strings such as ZA, TVM and ACF. Custom errors are cheaper to deploy and to revert with, and they carry a readable name in tooling, which removes the need for the reader to decode abbreviations.

Appendix · Metadata

Contract	FermiSwapper
Address	0xb1076fE3AB5e28005C7c323Bac5AC06a680d452e
Chain	Ethereum
Compiler	v0.8.28+commit.7893614a
Audit ID	independent-fermiswap
Date	10/11/2026

Files analyzed

```
[user] contracts/FermiSwapper.sol
[user] contracts/BaseManager.sol
[user] contracts/interfaces/IFermiEngine.sol
[user] contracts/interfaces/IFermiSwapper.sol
[user] contracts/interfaces/IFermiSwapCallback.sol
```

MANUAL AUDIT UPGRADE

Need a deeper audit?

$5,000

Pay in ETH, SOL, or USDT. Detailed manual review by Crypto Hawking, ETHDenver 2025 Winner. Comprehensive findings, remediation steps, and post-fix re-review. Report delivered in 3 business days.

Request Manual Audit ’

https://cryptohawking.com/audit/manual?ref=independent-fermiswap

Need it built, not just audited?

I also build production smart contracts end to end, token launches, DeFi protocols, staking, and full DEX stacks (v2, v3 concentrated liquidity, and beyond), plus fractional CTO engagements for Web3 teams. Fixes for the findings in this report can be scoped in the same engagement.

https://cryptohawking.com/services

WhatsApp (preferred): +92 322 4274236, https://wa.me/923224274236